Computers and Mathematics with Applications 213 (2026) 242–256

Contents lists available at ScienceDirect

Computers and Mathematics with Applications

journal homepage: www.elsevier.com/locate/camwa

A novel adaptive gradient-enhanced physics-informed neural networks for solving partial differential equations

Xun Yang, Maohua Ran *, Haodong Pu

School of Mathematical Sciences, Sichuan Normal University, Chengdu, 610066, China

ARTICLE INFO

Keywords:
Adaptive sampling framework
Partial differential equations
Gradient information
Physics-informed neural network

ABSTRACT

This paper proposes an adaptive sampling framework for gradient-enhanced physics-informed neural networks, designed to address the limitations of traditional physics-informed neural networks in resolving problems with sharp gradients. By integrating real-time gradient information, the framework dynamically identifies regions where approximate solutions to partial differential equations change rapidly. The coordination points within these critical regions are then adaptively refined to improve numerical performance. Additionally, a periodic resampling strategy is implemented on the initial training set after predefined iterations to better capture the overall behavior of the solution across the computational domain. Numerical experiments validate the superiority of the proposed method in accuracy and robustness for solving partial differential equations with sharp gradients.

1. Introduction

In recent years, deep learning methods, particularly deep neural networks (DNNs), have demonstrated remarkable success across diverse domains such as speech recognition, natural language processing, and image analysis [1–6]. Building upon these advancements, physics-informed neural networks (PINNs) have emerged as an innovative paradigm for solving both forward and inverse problems of partial differential equations (PDEs) [7]. Unlike conventional numerical PDEs solvers, PINNs eliminate the need for mesh discretization by directly embedding PDEs into the neural network's loss function. This meshless framework exhibits versatility in handling various PDEs types, including fractional differential equations, integro-differential equations, and stochastic differential equations [8–13], while demonstrating superior performance over traditional methods in high-dimensional problems. In addition, PINNs is widely used in many fields, including fluid mechanics [14–16], hydrogeophysics [17], and porous medium transport phenomena [18], and optics [19,20].

Despite their promise, PINNs face challenges in balancing accuracy and computational efficiency. Recent research has focused on adaptive strategies to address these limitations. For instance, Yu et al. Gradient-enhanced PINNs (gPINNs) are proposed, incorporating PDEs gradient terms into the loss function [21], which incorporate PDEs gradient terms into the loss function. Wu et al. [22] proposed a residual-based adaptive distribution sampling (RAD) method that dynamically adjusts residual point locations through nonlinear probability density functions. Following a similar principle. Wang et al. [23] introduced a multi-scale architecture for PINNs (M-PINNs), enhanced with residual blocks. Coupled with an adaptive sampling method that refines sampling locations according to the prevailing residual errors, this approach effectively captures high-frequency oscillatory components in PDEs solutions. Mao et al. [24] introduced gradient-driven adaptive sampling, prioritizing regions with large residuals and gradient magnitudes.

* Corresponding author.
E-mail addresses: sicnuyangxun@163.com (X. Yang), maohuaran@163.com (M. Ran), puhaodong10@163.com (H. Pu).

https://doi.org/10.1016/j.camwa.2026.04.026
Received 31 March 2025; Received in revised form 2 April 2026; Accepted 21 April 2026
Available online 29 April 2026

In PINNs, the loss function is a weighted sum of multiple loss terms from the PDEs, initial, and boundary conditions. Currently, weights are assigned to each loss term to minimize the error and enhance PINNs accuracy, see e.g., [25–31]. During training, each point has different importance, so different weights are given. These weights will be dynamically adjusted as training progresses, see e.g., [32–34]. For large-scale time-dependent PDEs, domain decomposition strategies accelerate training and improve accuracy [35–37]. Furthermore, Wu et al. [38] proposed deep fuzzy PINNs to further refine solution fidelity.

In this work, we propose a novel gradient-enhanced PINNs framework that integrates real-time gradient analysis and adaptive resampling, which we refer to as adaptive sampling gradient-enhanced physics-informed neural networks (ASGPINNs). The main innovations of ASGPINNs include:

- Gradient-driven refinement: After specified iterations, points with the largest gradient modes are identified from random samples, and training point density is increased locally.
- Dynamic resampling: The initial training set is periodically resampled at predefined intervals to enhance spatial coverage without increasing point count.
- Residual gradient augmentation: A gradient penalty term is added to the loss function to regularize sharp variations.

Numerical experiments confirm that our method outperforms both standard PINNs and existing gPINNs in solution accuracy.

The remainder of this paper is organized as follows: Section 2 details the PINNs framework and our gradient-resampling enhancements. Section 3 presents comparative numerical results, and Section 4 concludes the study.

2. Framework of PINNs and enhanced methods

This section provides a concise overview of the application of PINNs in solving PDEs and further presents the ASGPINNs.

2.1. PINNs for solving PDEs

This subsection briefly reviews the application of PINNs in solving PDEs.

Consider the following PDEs system:

$$\mathcal{F}[u](x,t) = f(x,t), \quad x \in \Omega,\ t \in [0,T], \tag{2.1}$$

$$u(x,0) = h(x,0), \quad x \in \Omega, \tag{2.2}$$

$$\mathcal{B}[u](x,t) = b(x,t), \quad x \in \partial\Omega,\ t \in [0,T], \tag{2.3}$$

where (2.1) governs the evolution of the system over time within the domain Ω, $\dim(\Omega) = d-1$. Here, $\mathcal{F}[u](x,t)$ comprises the solution $u(x,t)$ and its derivatives with respect to the spatial variable x and time variable t. Eqs. (2.2) and (2.3) represent the initial condition and the boundary condition, respectively, defining the initial state and the boundary behavior of the system.

Within the PINNs framework, a fully connected neural network approximates the PDEs solution. The network output $\hat{u}(x,t,\theta)$ serves as the surrogate solution, and its accuracy is improved by optimizing the parameters θ through the following minimization problem:

$$u(x,t) \approx \arg\min_{\hat{u}\in F} J(\hat{u}), \tag{2.4}$$

where

$$F := \{\hat{u}(\cdot,\cdot,\theta) : \theta \in \mathbb{R}^n\} \subset C(\bar{\Omega}\times[0,T]), \tag{2.5}$$

n denotes the number of parameters. The loss function $J(\hat{u})$ is composed of three key components:

$$J(\hat{u}) = \omega_r \mathcal{L}_r(\hat{u}) + \omega_0 \mathcal{L}_0(\hat{u}) + \omega_b \mathcal{L}_b(\hat{u}), \tag{2.6}$$

where ω_r, ω_b, and ω_0 are the weight coefficients assigned to the PDEs residual, boundary condition, and initial condition terms, respectively. The adjustment of these weights serves to balance the contribution of each term during training to enhance the accuracy and reliability of the surrogate solution. The specific forms of the loss terms $\mathcal{L}_r(\hat{u})$, $\mathcal{L}_0(\hat{u})$, and $\mathcal{L}_b(\hat{u})$ take the following specific forms:

$$\mathcal{L}_r(\hat{u}) = \frac{1}{N_r}\sum_{j=1}^{N_r}\left|\mathcal{F}[\hat{u}](x_r^j,t_r^j) - f(x_r^j,t_r^j)\right|^2, \quad (x_r^j,t_r^j) \in \Omega\times[0,T], \tag{2.7}$$

$$\mathcal{L}_0(\hat{u}) = \frac{1}{N_0}\sum_{j=1}^{N_0}\left|\hat{u}(x_0^j,0) - h(x_0^j,0)\right|^2, \quad x_0^j \in \Omega, \tag{2.8}$$

$$\mathcal{L}_b(\hat{u}) = \frac{1}{N_b}\sum_{j=1}^{N_b}\left|\mathcal{B}[\hat{u}](x_b^j,t_b^j) - b(x_b^j,t_b^j)\right|^2, \quad (x_b^j,t_b^j) \in \partial\Omega\times[0,T], \tag{2.9}$$

where N_r, N_b, and N_0 denote the number of collocation points corresponding to the domains $\Omega\times[0,T]$, $\partial\Omega\times[0,T]$, and Ω, respectively.

2.2. ASGPINNs for solving PDEs

To enhance solution accuracy and stability, the proposed ASGPINNs incorporate residual gradient information into the loss function:

$$J(\hat{u}) = \omega_r \mathcal{L}_r(\hat{u}) + \omega_b \mathcal{L}_b(\hat{u}) + \omega_0 \mathcal{L}_0(\hat{u}) + \sum_{i=1}^{d} w_{g_i} \mathcal{L}_{g_i}(\hat{u}), \tag{2.10}$$

where

$$\mathcal{L}_{g_i}(\hat{u}) = \frac{1}{N_r} \sum_{j=1}^{N_r} \left| \frac{\partial (\mathcal{F}[\hat{u}]\left(x_r^j, t_r^j\right) - f(x_r^j, t_r^j))}{\partial x_i} \right|^2, \quad (x_r^j, t_r^j) \in \Omega \times [0, T]. \tag{2.11}$$

In (2.10), the weight w_{g_i} of the gradient penalty term (where $i = d$ corresponds to the temporal partial derivative) is determined empirically to balance the contributions of all terms without compromising training stability. Following the standard practice for gradient-enhanced PINNs [21], we assign relatively small, often equal, values to w_{g_i} to prevent the gradient term from dominating the overall loss. These weights are further tuned via preliminary experiments on each test case to achieve an optimal balance among the residual, boundary condition, initial condition, and gradient terms.

Gradients characterize the rate of functional variation. Regions with large gradients–where the solution exhibits sharp transitions or features prone to error–receive higher sampling priority. The central aspect of the method is a gradient-informed adaptive sampling strategy designed to improve the approximation of the PDEs solution.

Specifically, the proposed ASGPINNs algorithm (Algorithm 1) dynamically refines the distribution of training points during network optimization. At predefined intervals (every m training steps until a maximum of E epochs), a candidate set of points S is randomly sampled across the computational domain. The gradient modulus $|\nabla \hat{u}(x)|$ of the current neural-network surrogate is evaluated at all points in S to identify regions of steep solution variation. The P points with the largest gradient magnitudes are then selected as key points $\mathcal{P}$, corresponding to regions where the solution changes most rapidly.

Algorithm 1 ASGPINNs algorithm.

Input: Initial residual training set M, boundary training set, initial value training set, Maximum number of training iterations E, ω_r, ω_b, ω_0, m, P, N, K,
Output: Trained neural network model
Initialize neural network parameters θ
$\mathcal{T}_f \leftarrow M$ ▷ Initial training set
for epoch = 1 to E **do**
  Compute loss: $J(\hat{u}) = \omega_r \mathcal{L}_r(\hat{u}) + \omega_b \mathcal{L}_b(\hat{u}) + \omega_0 \mathcal{L}_0(\hat{u}) + \sum_{i=1}^{d} w_{g_i} \mathcal{L}_{g_i}(\hat{u})$,
  Update θ using gradient descent
  if epoch $<$ E **and** $epoch \mod m = 0$ **then**
    Generate candidate set S
    Compute gradient $|\nabla \hat{u}(x)|$ for all $x \in S$
    Select P points with largest gradient magnitude as key points $\mathcal{P}$
    Within the neighbourhood of the key point set $\mathcal{P}$, generate a refined point set $\mathcal{H}$ such that $|\mathcal{H}| = N$.
    $\mathcal{T}_f \leftarrow \mathcal{T}_f \cup \mathcal{H}$ ▷ Add new points to training set
  end if
  if $epoch \mod K = 0$ **then**
    Resample initial set M' with same cardinality as M
    $\mathcal{T}_f \leftarrow M' \cup (\mathcal{T}_f \setminus M)$ ▷ Replace initial points
    $M \leftarrow M'$
  end if
end for

Within a local neighborhood of each key point, a refined set of points $\mathcal{H}$ is generated through systematic perturbation, thereby increasing the sampling density in these critical regions. These points are subsequently added to the residual training set $\mathcal{T}_f$, directing the computational effort of the network toward resolving challenging solution features.

Concurrently, to ensure broad exploration of the domain, the initial residual training set M is periodically resampled every K training steps while maintaining its original size. This strategy mitigates overfitting to a static initial point configuration and promotes comprehensive coverage of the solution domain throughout training.

The combined methodology–local refinement near high-gradient regions and periodic global resampling–facilitates a dynamic adjustment of the point distribution. This balances focused resolution of critical features with extensive exploration of the entire domain. As the numerical experiments demonstrate, this adaptive sampling approach enhances the reliability of the solution while preserving computational efficiency.

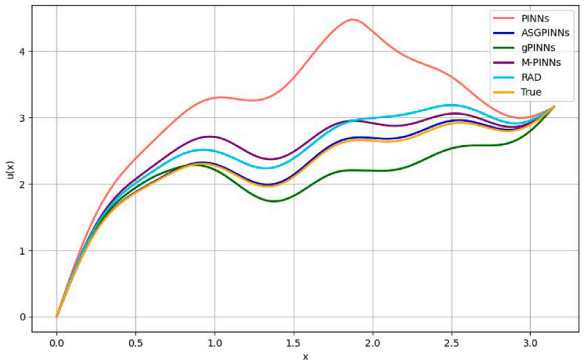


Fig. 1. Numerical solutions for the 1D Poisson equation.

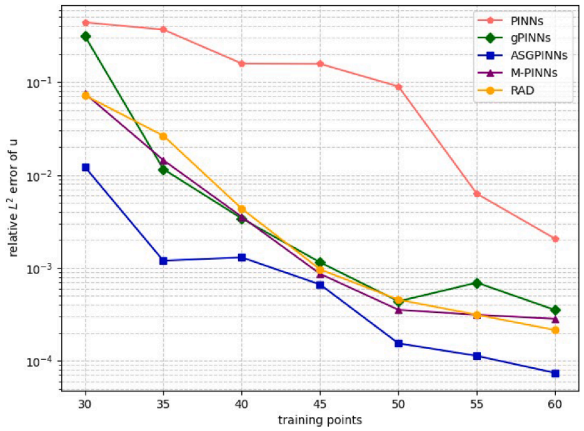


Fig. 2. Relative L^2 errors versus number of training points (1D Poisson equation).

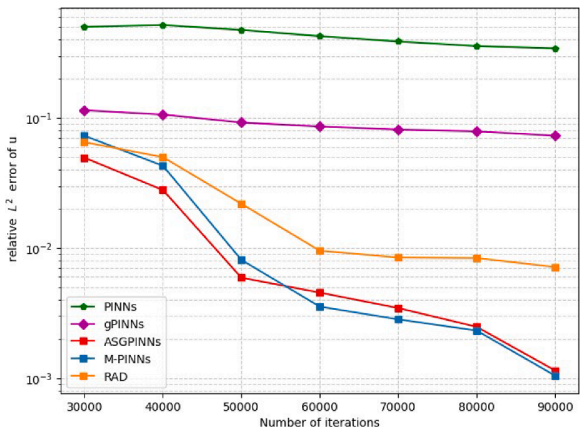
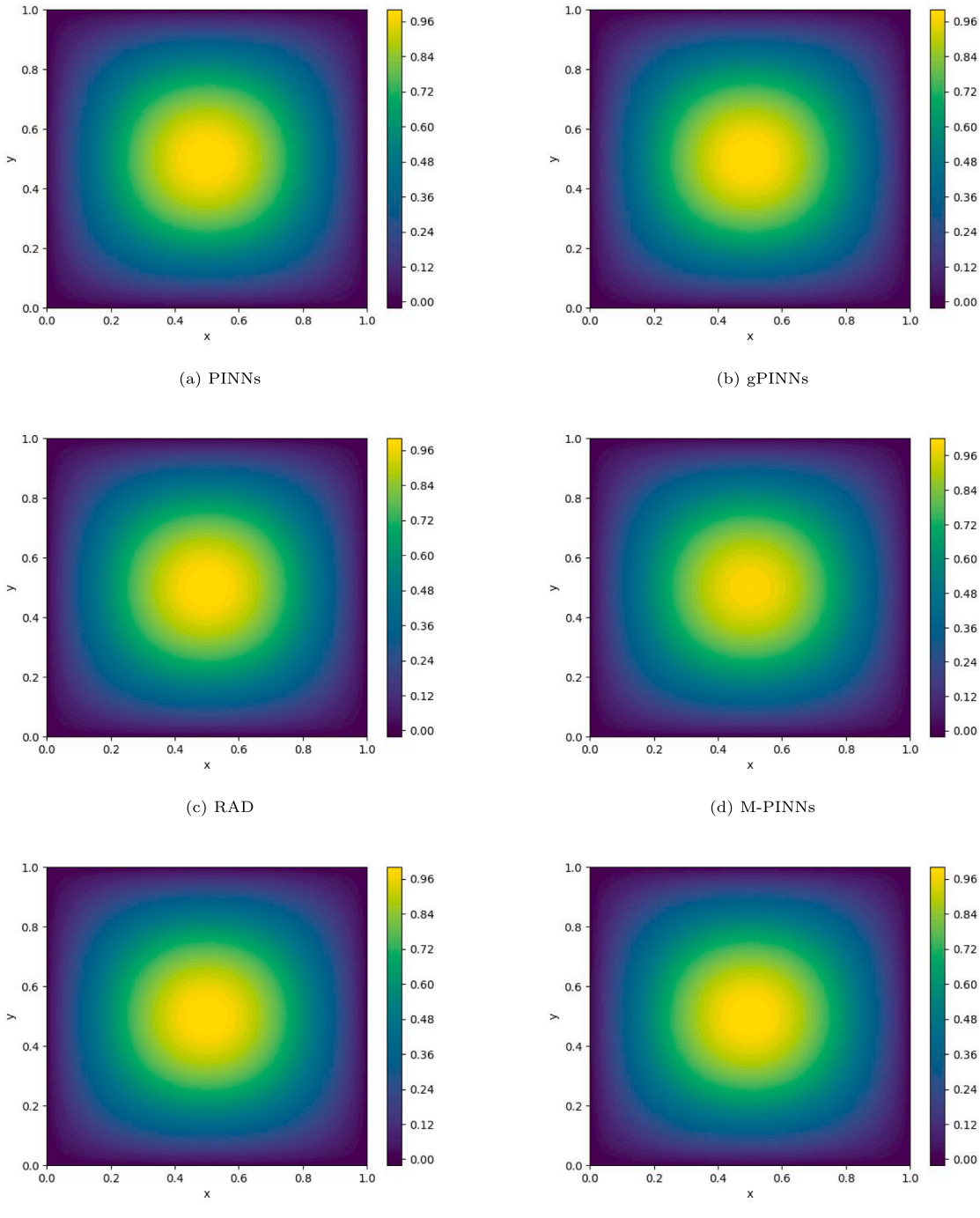


Fig. 3. Convergence of relative L^2 errors during training (1D Poisson equation).

3. Numerical experiments

3.1. Experimental settings

This section aims to evaluate the proposed ASGPINNs method by comparing its results obtained from solving partial differential equations with those from standard PINNs, gPINNs, RAD, and M-PINNs. To ensure the fairness of comparison, all methods adhere to an identical adaptive training schedule: specifically, both RAD and M-PINNs update their global collocation point sets every m training steps, a practice that aligns with the periodic sampling strategy employed in ASGPINNs.



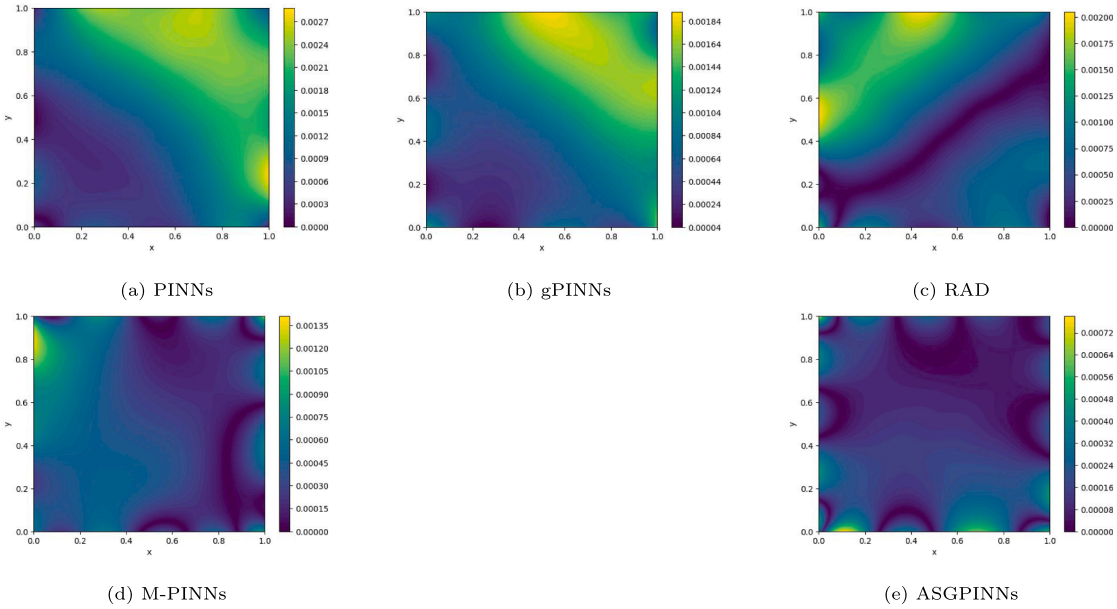
(a) PINNs (b) gPINNs

(c) RAD (d) M-PINNs

(e) ASGPINNs (f) Exact

Fig. 4. Exact and numerical solutions for the 2D Poisson equation.

In all experiments, the tanh function is used as the activation function. While M-PINNs utilizes a Residual Neural Network architecture (with specific details provided in each respective experiment), the depth and width of its networks remain consistent with the parameters listed in Table 1. The key hyperparameters for all test cases are summarized in Table 1. Additionally, the method-specific parameters for RAD and M-PINNs are set to $c = k = 1$, following the default values in their original publications. To maintain comparability across experiments, the loss function weights for PINNs, gPINNs, M-PINNs, and RAD are set equal to their corresponding weights within the ASGPINNs framework.



(a) PINNs (b) gPINNs (c) RAD

(d) M-PINNs (e) ASGPINNs

Fig. 5. Absolute error distributions for the 2D Poisson equation.

Table 1
Hyperparameter configurations for all test cases.

PDEs	Depth	Width	Optimizer	Learning rate	Iterations	m	P	N	K
1D Poisson	4	40	Adam	0.001	15,000	3000	1	2	5000
2D Poisson	4	40	Adam	0.001	20,000	4000	4	40	4000
Diffusion	4	40	Adam	0.001	20,000	1000	1	10	4000
Burgers'	3	50	Adam	0.001	20,000	1000	10	20	3000
3D Poisson	4	32	Adam	0.001	20,000	4000	50	500	6000

Numerical accuracy is assessed using the relative L^2 error, defined as follows:

$$\text{Error} = \frac{\sqrt{\sum_{j=1}^{N_1} |\hat{u}(x_j, t_j) - u_{\text{ref}}(x_j, t_j)|^2}}{\sqrt{\sum_{j=1}^{N_1} |u_{\text{ref}}(x_j, t_j)|^2}},$$

where $\hat{u}$ represents the predicted solution, u_{ref} represents the reference (or exact) solution, and N_1 denotes the total number of test points.

All experiments were conducted on the same hardware platform: a Lenovo XiaoXin Air 15 ALC 2021 laptop equipped with an AMD Ryzen 7 5700U processor, 16 GB of RAM, and integrated AMD Radeon Vega 8 graphics.

3.2. 1D Poisson equation

$$\begin{cases} -u''(x) = 8\sin(8x) + \sum_{j=1}^{4} j\sin(jx), \quad x \in [0, \pi], & (3.1) \\ u(0) = 0, \quad u(\pi) = \pi. & (3.2) \end{cases}$$

The exact solution corresponding to this boundary value problem is:

$$u(x) = x + \frac{\sin(8x)}{8} + \sum_{j=1}^{4} \frac{\sin(jx)}{j}, \quad x \in [0, \pi].$$

For this case, the loss weights are set as $\omega_r = 1$, $\omega_b = 1$, and $\omega_g = 0.01$. The methods compared include PINNs, gPINNs, M-PINNs, RAD, and ASGPINNs. Among these, the architecture for M-PINNs incorporates two residual blocks with scale factors (1, 2, 3), each block consisting of two hidden layers with 40 neurons per layer. Regarding training configurations, PINNs, gPINNs, RAD, and M-PINNs

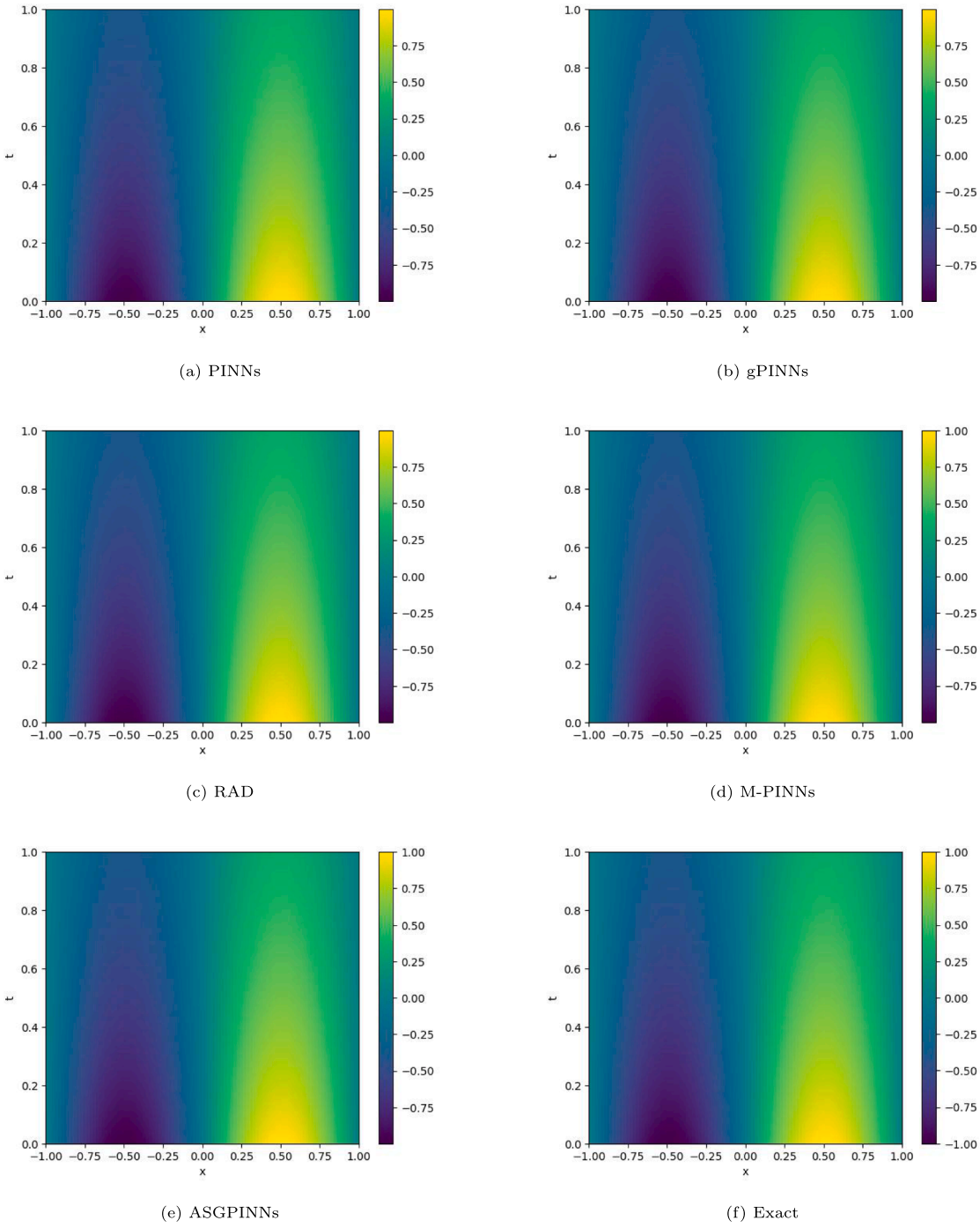
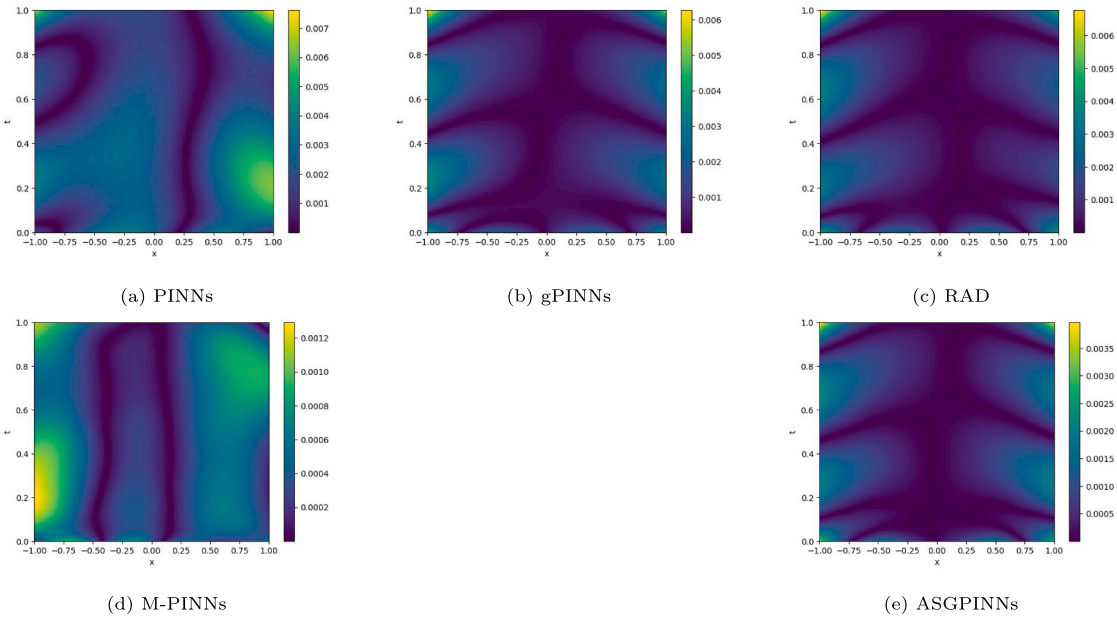


Fig. 6. Exact and computed solutions for the Diffusion equation.

all employ 30 residual points. In contrast, ASGPINNs commence with 22 initial points, adaptively adding points through periodic policy updates until a total of 30 points is attained. The relevant numerical results are presented in Fig. 1 and Fig. 3.

Fig. 1 compares the numerical solutions from each method against the exact solution. It can be observed that under identical training settings, ASGPINNs demonstrates superior approximation accuracy. Table 2 further summarizes the computational time and errors for each method. From the data in the table, PINNs achieves the shortest runtime, while M-PINNs requires significantly longer computational time. In terms of accuracy, ASGPINNs attains the highest level among all compared methods.



(a) PINNs (b) gPINNs (c) RAD

(d) M-PINNs (e) ASGPINNs

Fig. 7. Point-wise absolute errors for the Diffusion equation.

Table 2
Errors and computational time for the 1D Poisson equation.

Methods	PINNs	gPINNs	RAD	M-PINNs	ASGPINNs
Error	4.39×10^{-1}	3.12×10^{-1}	7.19×10^{-2}	7.52×10^{-2}	1.22×10^{-2}
Time (s)	61.5	114.0	65.6	711.9	122.3

In addition, Fig. 2 illustrates the relationship between the relative L^2 error and the number of training points. In this comparison, ASGPINNs consistently maintains the lowest error level. Fig. 3 then presents the convergence behavior of the error during the training iterations. In the early stages of training, ASGPINNs exhibits the fastest convergence rate. As training progresses, M-PINNs shows a slight advantage in convergence speed, a phenomenon that may be attributed to its residual network structure, which helps mitigate overfitting.

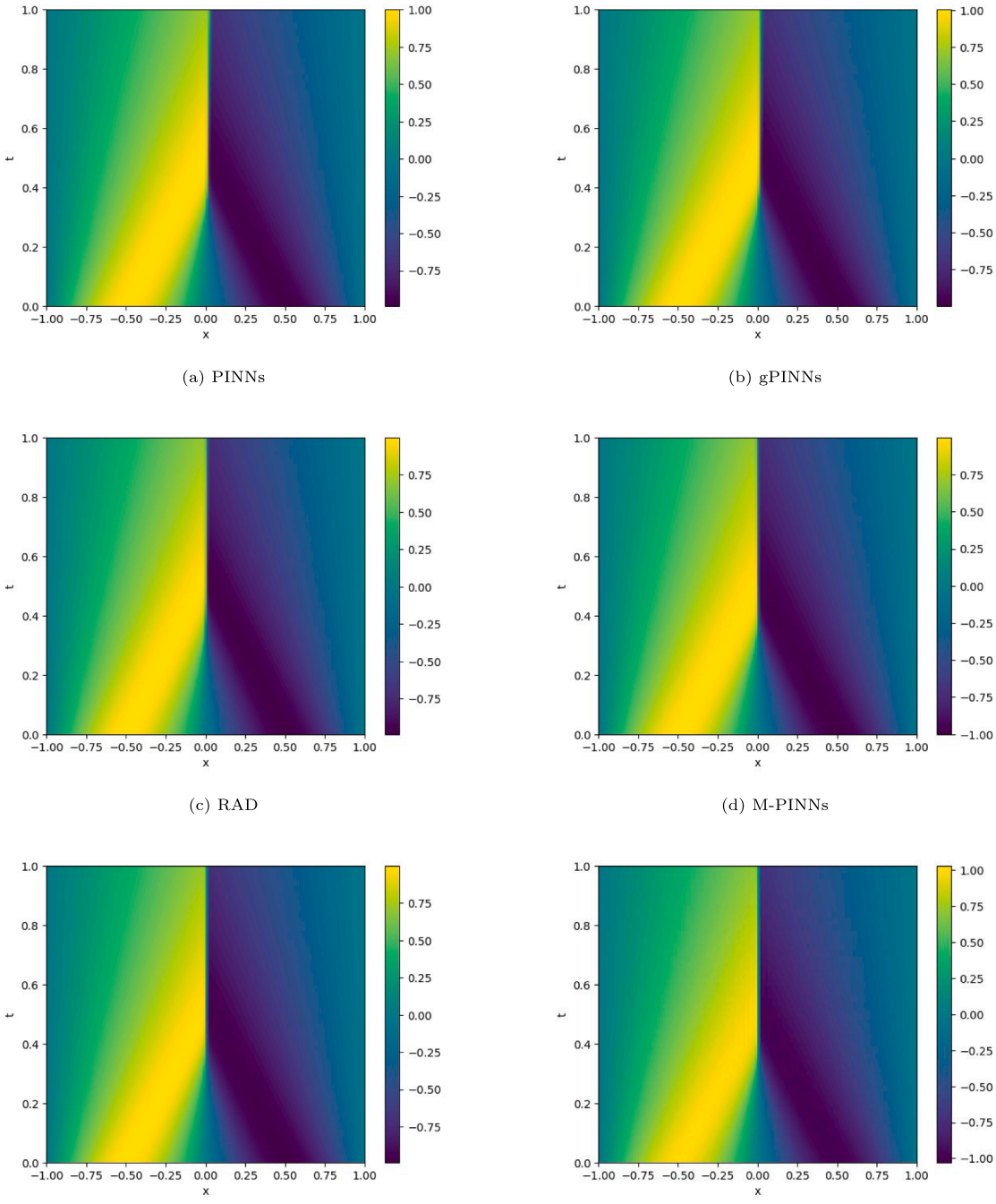
3.3. 2D Poisson equation

$$\begin{cases} -\nabla^2 u = 2\pi^2 \sin(\pi x)\sin(\pi y), \quad (x, y) \in [0, 1]^2, & (3.3) \\ u(0, y) = u(1, y) = 0, \quad y \in [0, 1], & (3.4) \\ u(x, 0) = u(x, 1) = 0, \quad x \in [0, 1]. & (3.5) \end{cases}$$

For $(x, y) \in [0, 1]^2$, the analytical solution to this problem is $u(x, y) = \sin(\pi x)\sin(\pi y)$.

In this case, the loss weights are configured as $\omega_r = 0.01$, $\omega_b = 1$, $\omega_{g_1} = \omega_{g_2} = 10^{-4}$. The M-PINNs scheme employs a multi-scale residual network with four scale factors $(1, 2, 3, 4)$, and the main body of the network consists of two residual blocks, each containing two hidden layers with 40 neurons. Regarding training data, PINNs, gPINNs, RAD, and M-PINNs all use 1000 residual collocation points and 400 boundary points. In contrast, ASGPINNs begins with an initial setup of 840 residual collocation points and 400 boundary points, and adaptively increases the residual points during training until the total reaches 1000.

A comparison of numerical results is shown in Fig. 4, which contrasts the predicted solutions from each method with the exact solution. Fig. 5 then presents the corresponding spatial distribution of absolute errors. According to the quantitative analysis in Table 3, ASGPINNs achieves the smallest relative L^2 error, outperforming the other methods. However, when considering computational efficiency, PINNs, gPINNs, and RAD all complete training in less time than ASGPINNs, with M-PINNs requiring the longest computational time.



(a) PINNs (b) gPINNs

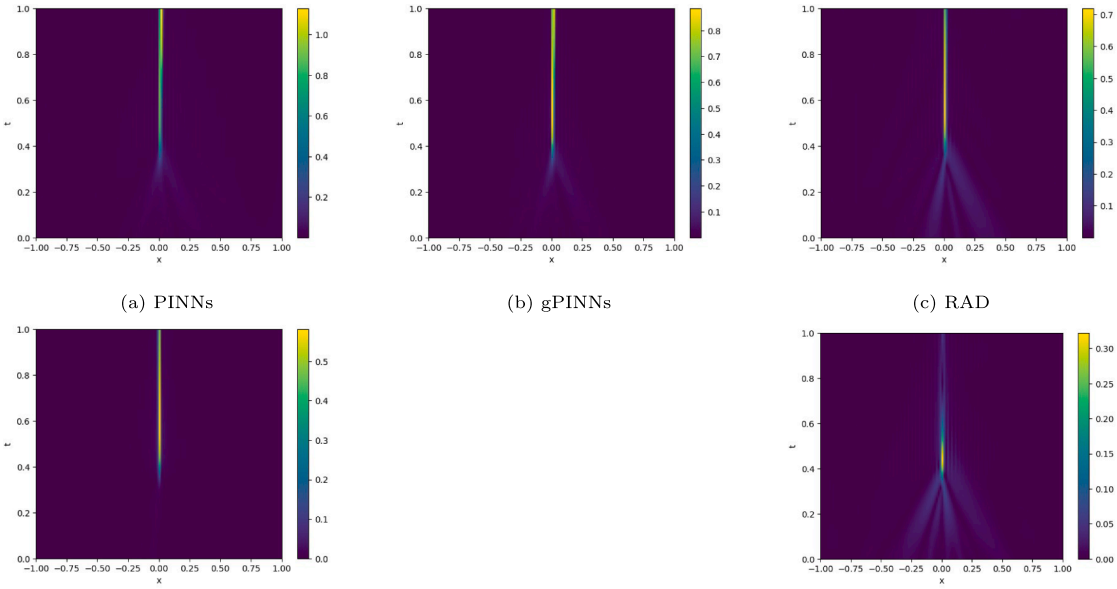
(c) RAD (d) M-PINNs

(e) ASGPINNs (f) Reference

Fig. 8. Numerical versus reference solutions for Burgers' equation.

Table 3
Relative L^2 error and training time for the 2D Poisson equation.

Methods	PINNs	gPINNs	RAD	M-PINNs	ASGPINNs
Error	3.14×10^{-3}	2.09×10^{-3}	1.82×10^{-3}	8.31×10^{-4}	3.36×10^{-4}
Time (s)	240.6	928.0	253.1	3937.8	948.5



(a) PINNs (b) gPINNs (c) RAD

(d) M-PINNs (e) ASGPINNs

Fig. 9. Point-wise absolute errors for Burgers' equation.

Table 4
Relative L^2 errors and training times for the Diffusion equation.

Methods	PINNs	gPINNs	RAD	M-PINNs	ASGPINNs
Error	5.11×10^{-3}	2.45×10^{-3}	2.39×10^{-3}	1.08×10^{-3}	1.66×10^{-3}
Time (s)	292.2	1121.0	293.5	3096.2	1081.4

3.4. Diffusion equation

$$\begin{cases} u_t = u_{xx} + \exp(-t)\big(\pi^2 \sin(\pi x) - \sin(\pi x)\big), \quad x \in [-1, 1],\ t \in [0, 1], & (3.6) \\ u(-1, t) = u(1, t) = 0, \quad t \in [0, 1], & (3.7) \\ u(x, 0) = \sin(\pi x), \quad x \in [-1, 1]. & (3.8) \end{cases}$$

The exact solution is given by:

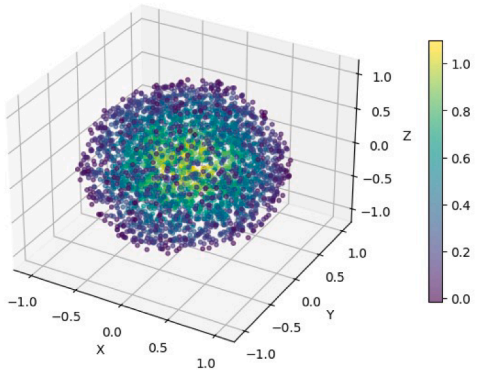
$$u(x, t) = \exp(-t)\sin(\pi x), \qquad x \in [-1, 1],\ t \in [0, 1].$$

For this example, the weighting scheme is set as $\omega_r = 1$, $\omega_b = 1$, $\omega_0 = 1$, $\omega_{g_1} = 10^{-5}$, and $\omega_{g_2} = 10^{-5}$. The M-PINNs scheme employs a multi-scale residual network with four scale factors $(1, 2, 3, 4)$. Its architecture comprises two residual blocks, each containing two hidden layers of 40 neurons. PINNs, gPINNs, RAD, and M-PINNs are trained with 2000 residual points, 200 boundary points, and 100 initial points. In comparison, the ASGPINNs method starts with $1,810$ residual points, 200 boundary points, and 100 initial points, then adaptively adds residual points until the total reaches $2,000$.

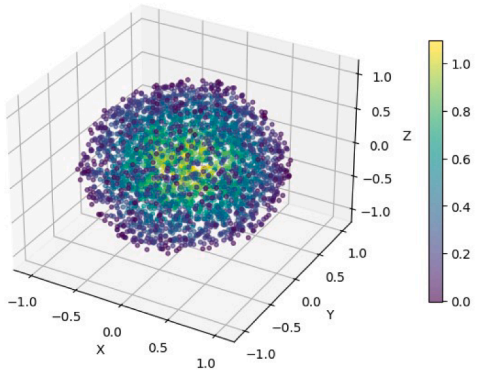
Figs. 6 and 7 compare the numerical solutions and visualize the absolute errors of the five methods, respectively. Table 4 summarizes the corresponding computational times and relative L^2 errors. The results indicate that PINNs achieve the shortest runtime but the lowest accuracy, with a relative L^2 error of 5.11×10^{-3}. M-PINNs obtain the highest accuracy (1.08×10^{-3}) at a substantially increased computational cost. By contrast, ASGPINNs attain the second-highest accuracy (1.66×10^{-3}), only slightly below that of M-PINNs, while requiring significantly less computational time.

3.5. Burgers' equation

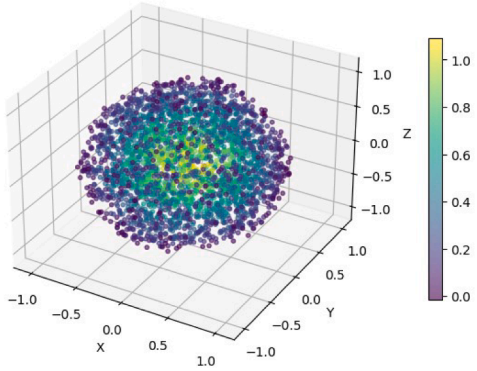
$$\begin{cases} u_t + uu_x - vu_{xx} = 0, \quad x \in [-1, 1],\ t \in [0, 1], & (3.9) \\ u(-1, t) = u(1, t) = 0, \quad t \in [0, 1], & (3.10) \\ u(x, 0) = -\sin(\pi x), \quad x \in [-1, 1], & (3.11) \end{cases}$$



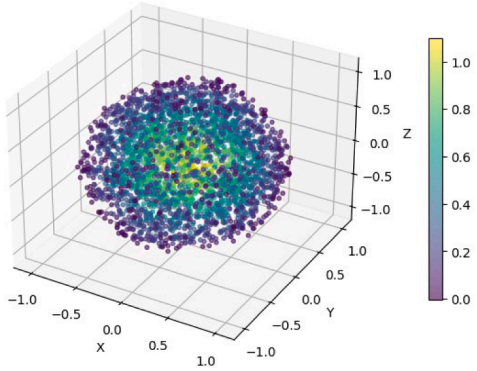
(a) PINNs



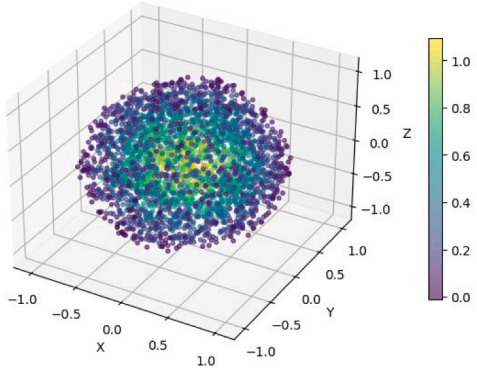
(b) gPINNs



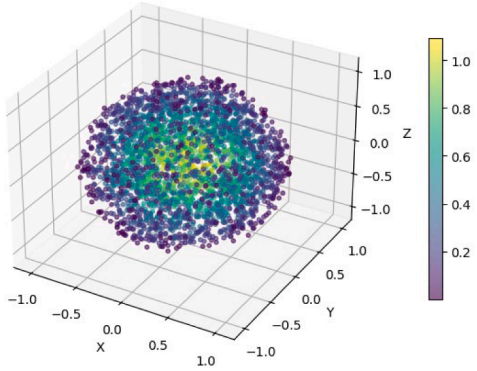
(c) RAD



(d) M-PINNs



(e) ASGPINNs

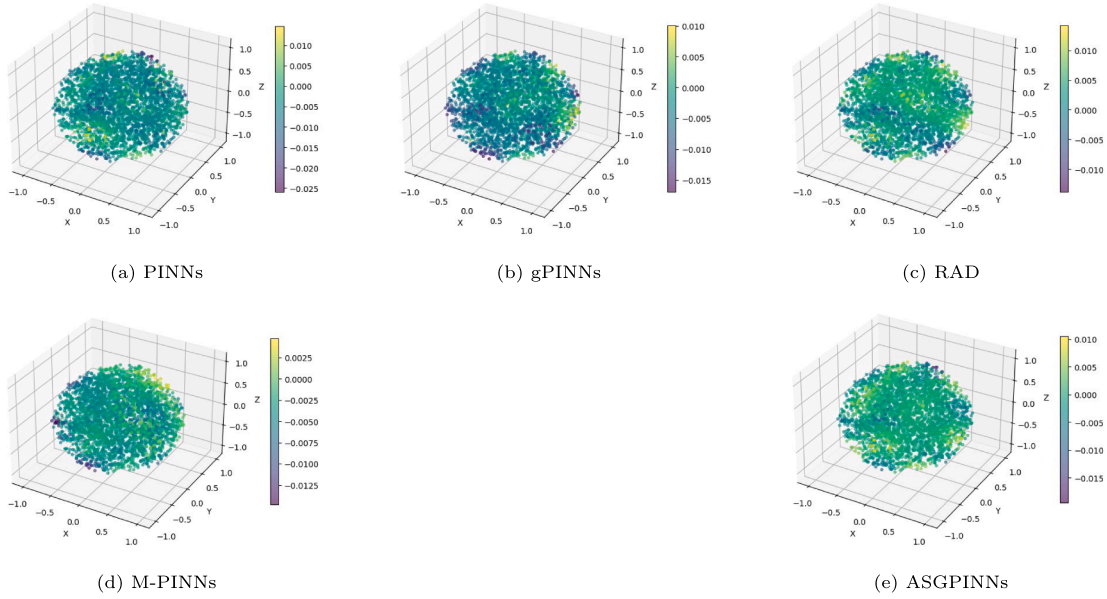


(f) Exact

Fig. 10. Numerical and exact solutions for the 3D Poisson equation.

where $\upsilon = 0.01/\pi$. Reference solutions are generated using a Fourier pseudo-spectral method with spatial discretization $\Delta x = 1/128$ and a fourth-order Runge-Kutta time-stepping scheme ($\Delta t = 10^{-4}$).

For this example, the loss weights are fixed as $\omega_r = 1$, $\omega_b = 1$, $\omega_0 = 5$, $\omega_{g_1} = 10^{-5}$, and $\omega_{g_2} = 10^{-5}$. The M-PINNs scheme employs a multi-scale residual network with three scale factors $(1, 2, 3)$. The network architecture consists of two residual blocks: the first block contains two hidden layers and the second block contains one hidden layer, each layer having 50 neurons. PINNs, gPINNs, RAD, and M-PINNs are all trained with $4,000$ residual points, 200 boundary points, and 100 initial points. In contrast, the ASGPINNs method



(a) PINNs (b) gPINNs (c) RAD

(d) M-PINNs (e) ASGPINNs

Fig. 11. Absolute errors between numerical and exact solutions for the 3D Poisson equation.

Table 5
Relative L^2 errors and training times for Burgers' equation.

Methods	PINNs	gPINNs	RAD	M-PINNs	ASGPINNs
Error	1.28×10^{-1}	1.11×10^{-1}	8.01×10^{-2}	6.18×10^{-2}	2.40×10^{-2}
Time (s)	437.0	1171.4	462.9	3583.7	990.6

begins with $3,620$ residual points, 200 boundary points, and 60 initial points, then adaptively adds residual points until the total reaches $4,000$.

Figs. 8 and 9 display the numerical solutions and the corresponding absolute-error distributions for the five methods, respectively. Table 5 summarizes the computation times and relative L^2 errors. The results show that PINNs have the shortest computation time but the lowest accuracy, with a relative L^2 error of 1.28×10^{-1}. The ASGPINNs method achieves the highest accuracy (relative L^2 error of 2.40×10^{-2}). Its computation time lies between those of PINNs/RAD and gPINNs/M-PINNs, and is notably lower than that of M-PINNs, which exhibits the longest runtime.

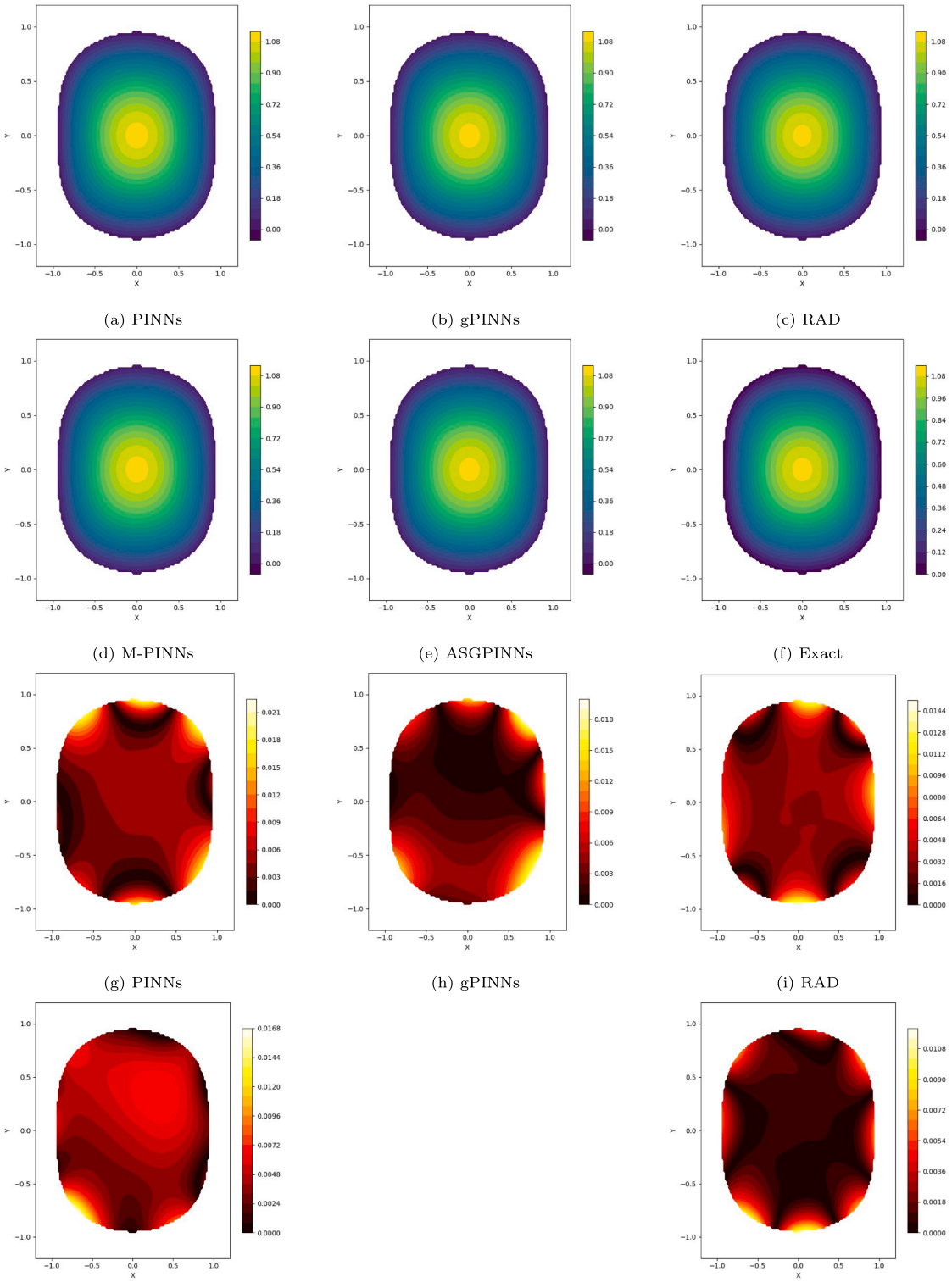
3.6. 3D Poisson equation

$$\begin{cases} -\nabla^2 u(x,y,z) = f(x,y,z), \quad (x,y,z)\in\Omega, & (3.12)\\ u(x,y,z) = 0, \quad (x,y,z)\in\partial\Omega, & (3.13)\end{cases}$$

where $f(x,y,z) = 6 + 5\cos(3x)\cos(4y)\cos(5z)$, $\Omega = \{(x,y,z) \mid \phi(x,y,z) < 0\}$, and $\phi(x,y,z) = x^2+y^2+z^2-1-0.1\cos(3x)\cos(4y)\cos(5z)$. The corresponding exact solution is $u(x,y,z) = 1+0.1\cos(3x)\cos(4y)\cos(5z)-(x^2+y^2+z^2)$.

For this example, the loss weights are set as $\omega_r = 1$, $\omega_b = 5$, $\omega_{g_1} = 10^{-3}$, $\omega_{g_2} = 10^{-3}$, and $\omega_{g_3} = 10^{-3}$. The M-PINNs scheme employs a multi-scale residual network with three scale factors $(1,2,3)$. The network consists of two residual blocks, each containing two hidden layers with 32 neurons per layer. PINNs, gPINNs, RAD, and M-PINNs are all trained with $10,000$ residual points and $1,000$ boundary points. In contrast, the ASGPINNs method starts with $8,000$ residual points and $1,000$ boundary points, then adaptively adds residual points until the total reaches $10,000$.

Fig. 10 compares the predicted solutions of the five methods against the exact solution. Fig. 11 presents the distribution of the corresponding absolute errors, while Fig. 12 displays heatmaps of the solutions at the cross-section $z = 0$ for each method, together with their pointwise absolute errors relative to the exact solution. Quantitative error analysis in Table 6 confirms that ASGPINNs achieve the smallest relative L^2 error, outperforming the other methods. In terms of computational efficiency, however, PINNs and RAD are faster than ASGPINNs, whereas gPINNs and M-PINNs are slower, with M-PINNs exhibiting the longest runtime.



(a) PINNs (b) gPINNs (c) RAD

(d) M-PINNs (e) ASGPINNs (f) Exact

(g) PINNs (h) gPINNs (i) RAD

(j) M-PINNs (k) ASGPINNs

Fig. 12. Numerical solutions, exact solution, and pointwise errors at the cross-section $z = 0$ for PINNs, gPINNs, RAD, M-PINNs, and ASGPINNs applied to the 3D Poisson equation.

Table 6
Relative L^2 errors and training times for the 3D Poisson equation.

Methods	PINNs	gPINNs	RAD	M-PINNs	ASGPINNs
Error	1.15×10^{-2}	9.65×10^{-3}	7.88×10^{-3}	9.20×10^{-3}	5.47×10^{-3}
Time (s)	1695.2	4599.1	1712.0	11762.3	3992.1

4. Conclusions

A novel adaptive adaptive sampling gradient-enhanced physics-informed neural networks is proposed. This approach effectively improves the accuracy of partial differential equation solutions by incorporating a gradient-aware mechanism and an adaptive sampling strategy. Numerical experiments demonstrate that, compared to existing PINNs [7], gPINNs [21], and RAD [22] methods, the proposed ASGPINNs exhibit superior accuracy. When compared with M-PINNs [23], the accuracy of ASGPINNs varies across different problems; based on the experiments presented, it achieves higher accuracy in the majority of test cases. In terms of computational efficiency, ASGPINNs require slightly more runtime than PINNs and RAD, comparable to gPINNs, but significantly less than M-PINNs, while outperforming the latter in overall accuracy. In summary, this method offers a new perspective for neural-network-based PDE solvers and further advances the development of numerical methods for nonlinear differential equations.

Code availability statement

The source code that supports this manuscript is publicly available in Zenodo at https://doi.org/10.5281/zenodo.19381986.

Data availability

No data was used for the research described in the article.

Ackonwledgments

This work is supported by the Sichuan Science and Technology Program (No. 2024NSFSC0441).

References

[1] M. Dissanayake, N. Phan-Thien, Neural-network-based approximations for solving partial differential equations, Commun. Numer. Methods Eng 10 (1994) 195–201.
[2] V. Ashish, S. Noam, P. Niki, U. Jakob, Attention is all you need, Adv. Neural Inf. Process, Syst. 30 (2017) 5998–6008.
[3] J.X. Gu, Z.H. Wang, J. Kuen, L.Y. Ma, Recent advances in convolutional neural networks, Pattern Recognit. 77 (2018) 354–377.
[4] Y. Lecun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (2015) 436–444.
[5] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, Commun. ACM 60 (2017) 84–90.
[6] G. Hinton, L. Deng, D. Yu, G.E. Dahl, Deep neural networks for acoustic modeling in speech recognition, IEEE Signal Process Mag 29 (2012) 82–97.
[7] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys 378 (2019) 686–707.
[8] G. Pang, L. Lu, G.E. Karniadakis, fPINNs: fractional physics-informed neural networks, SIAM J. Sci. Comput. 41 (2019) 2603–2626.
[9] L. Guo, H. Wu, X.C. Yu, T. Zhou, Monte Carlo fPINNs: deep learning method for forward and inverse problems involving high dimensional fractional partial differential equations, Comput. Methods Appl. Mech. Engrg. 400 (2022) 115523.
[10] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, DeepXDE: a deep learning library for solving differential equations, SIAM Rev. 63 (2021) 208–228.
[11] L. Yuan, Y.Q. Ni, X.Y. Deng, S. Hao, A-PINN: auxiliary physics informed neural networks for forward and inverse problems of nonlinear integro-differential equations, J. Comput. Phys 462 (2022) 111260.
[12] D. Zhang, L. Lu, L. Guo, G.E. Karniadakis, Quantifying total uncertainty in physics-informed neural networks for solving forward and inverse stochastic problems, J. Comput. Phys 397 (2019) 108850.
[13] D. Zhang, L. Guo, G.E. Karniadakis, Learning in modal space: solving time-dependent stochastic PDEs using physics-informed neural networks, SIAM J. Sci. Comput. 42 (2020) 639–665.
[14] M. Raissi, Z.C. Wang, M.S. Triantafyllou, G.E. Karniadakis, Deep learning of vortex-induced vibrations, J. Fluid Mech. 2019, pp. 119–137.
[15] M. Raissi, A. Yazdani, G.E. Karniadakis, Hidden fluid mechanics: learning velocity and pressure fields from flow visualizations, Science 367 (2020) 1026–1030.
[16] S.Z. Cai, Z.P. Mao, Z.C. Wang, M.L. Yin, Physics-informed neural networks (PINNs) for fluid mechanics: a review, Acta Mech. Sin. 37 (2021) 1727–1738.
[17] X.H. Meng, G.E. Karniadakis, A composite neural network that learns from multifidelity data: application to function approximation and inverse PDE problems, J. Comput. Phys 401 (2020) 109020.
[18] G.F. Pang, L. Lu, G.E. Karniadakis, FPINNs: fractional physics-informed neural networks, SIAM J. Sci. Comput. 41 (2019) 2603–2626.
[19] Y. Chen, L. Lu, G.E. Karniadakis, L.D. Negro, Physics-informed neural networks for inverse problems in nano-optics and metamaterials, Opt. Express 28 (2020) 11618–11633.
[20] L. Lu, R. Pestourie, W. Yao, Z. Wang, Physics-informed neural networks with hard constraints for inverse design, SIAM J. Sci. Comput. 43 (2021) 1105–1132.
[21] J. Yu, L. Lu, X.H. Meng, G.E. Karniadakis, Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems, Comput. Methods Appl. Mech. Engrg. 393 (2022) 114823.
[22] C.X. Wu, M. Zhu, Q.Y. Tan, Y. Kartha, A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks, Comput. Methods Appl. Mech. Engrg. 403 (2023) 115671.
[23] J. Wang, X.L. Feng, H. Xu, Adaptive sampling points based multi-scale residual network for solving partial differential equations, Comput. Math. Appl. 169 (2024) 223–236.
[24] Z.P. Mao, X.H. Meng, Physics-informed neural networks with residual/gradient-based adaptive sampling methods for solving partial differential equations with sharp solutions, Appl. Math. Mech. -Engl. Ed. 44 (2023) 1069–1084.
[25] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, SIAM J. Sci. Comput. 43 (2021) 3055–3081.

[26] C.L. Wight, J. Zhao, Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks, Commun. Comput. Phys. 29 (2021) 930–954.
[27] D. Liu, Y. Wang, A dual-dimer method for training physics-constrained neural networks with minimax architecture, Neural Netw. 136 (2021) 112–125.
[28] Z.X. Xiang, W. Peng, X. Liu, W. Yao, Self-adaptive loss balanced physics-informed neural networks, Neurocomputing 496 (2022) 11–34.
[29] S.J. Anagnostopoulos, J.D. Toscano, N. Stergiopulos, G.E. Karniadakis, Residual-based attention in physics-informed neural networks, Comput. Methods Appl. Mech. Engrg. 421 (2024) 116805.
[30] S. Wang, X. Yu, P. Perdikaris, When and why PINNs fail to train: a neural tangent kernel perspective, J. Comput. Phys 449 (2022) 110768.
[31] B. Gao, R.X. Yao, Y. Li, Physics-informed neural networks with adaptive loss weighting algorithm for solving partial differential equations, Comput. Math. Appl. 181 (2025) 216–227.
[32] L.D. Mcclenny, U.M. Braga-Neto, Self-adaptive physics-informed neural networks, J. Comput. Phys 474 (2023) 111722.
[33] Y.J. Song, H. Wang, H. Yang, M.L. Taccari, Loss-attentional physics-informed neural networks, J. Comput. Phys 501 (2024) 112781.
[34] Y.Q. Gu, H.Z. Yang, C. Zhou, Selectnet: self-paced learning for high-dimensional partial differential equations, J. Comput. Phys 441 (2021) 110444.
[35] K. Shukla, A.D. Jagtap, G.E. Karniadakis, Parallel physics-informed neural networks via domain decomposition, J. Comput. Phys 447 (2021) 110683.
[36] A.D. Jagtap, G.E. Karniadakis, Extended physics-informed neural networks (XPINNs): a generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations, Commun. Comput. Phys. 28 (2020) 2002–2041.
[37] X.H. Meng, Z. Li, D.K. Zhang, G.E. Karniadakis, PPINN: parareal physics-informed neural network for time-dependent pdes, Comput. Methods Appl. Mech. Engrg. 370 (2020) 113250.
[38] W.Y. Wu, S.Y. Duan, Y. Sun, Y. Yu, Deep fuzzy physics-informed neural networks for forward and inverse PDE problems Neural Netw. 181 (2025) 106750.